Factory Floor Production System - Technical Documentation

Table of Contents

System Overview

Purpose

The Factory Floor Production System is a real-time manufacturing execution system (MES) designed to track production orders, material consumption, and inventory movements in a manufacturing environment.

Key Features

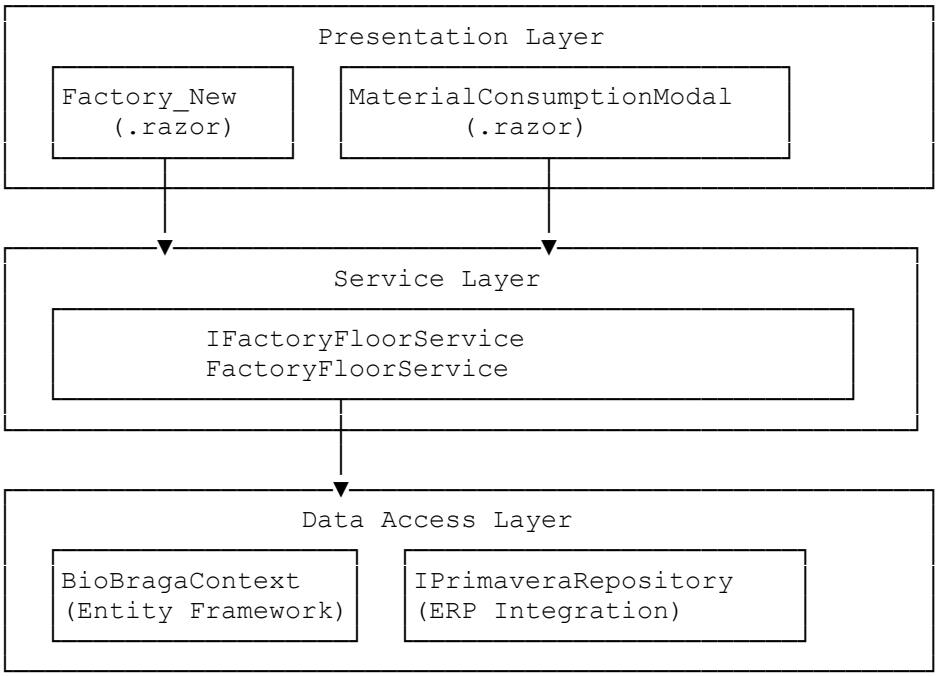
- Real-time production order management
- Material consumption tracking with variance analysis
- Automated stock movement generation
- Historical consumption pattern analysis
- Multi-machine production support
- Integration with Primavera ERP system

Technology Stack

- **Frontend:** Blazor Server (.NET 8)
- **UI Framework:** MudBlazor
- **Backend:** ASP.NET Core
- **Database:** Entity Framework Core
- **External Integration:** Primavera ERP

Architecture

System Architecture Diagram



Design Patterns

- **Repository Pattern:** Data access abstraction
- **Service Layer Pattern:** Business logic encapsulation
- **Dependency Injection:** IoC container for loose coupling
- **Unit of Work:** Transaction management via EF Core

Component Documentation

Factory_New.razor

Purpose

Main production floor interface providing real-time visualization of production orders and control mechanisms.

Component Structure

```
@page "/factory_new"
@page "/factory_new/{maquinaid}"
@layout FactoryLayout
```

Key Properties

Property	Type	Description
`maquinaid`	`string?`	Route parameter for machine filtering
`CurrentWorker`	`CrmUtilizador?`	Cascading parameter for authenticated worker
`allProductionOrders`	`List<OrdemProducao>`	Complete order collection
`filteredProductionOrders`	`List<OrdemProducao>`	Machine-filtered orders
`selectedMachine`	`CrmEquipamento?`	Currently selected machine
`todayStats`	`ProductionStatsDto?`	Today's production metrics

Lifecycle Methods

OnInitializedAsync()

```
protected override async Task OnInitializedAsync()
{
    await LoadInitialData();
    isDataLoaded = true;
}
```

- Loads machines and production orders
- Sets initial data loaded flag

OnParametersSetAsync()

```
protected override async Task OnParametersSetAsync()
```

- Handles URL parameter changes
- Manages browser navigation (back/forward)
- Updates filtered data based on machine selection

Key Methods

SelectMachine(string? machineId)

- Updates URL for bookmarkability
- Filters orders locally without server roundtrip
- Updates production statistics

RequestPinForAction(OrdemProducao order, ProductionAction action)

- Opens PIN authentication dialog
- Validates worker credentials
- Executes production action upon successful authentication

ExecuteProductionAction(OrdemProducao order, ProductionAction action)

- Processes Start/Pause/Stop actions
- Opens material consumption modal for Pause/Stop
- Updates order status via service layer

MaterialConsumptionModal.razor

Purpose

Modal dialog for recording actual material consumption and production quantities.

Key Features

- Dynamic material loading based on production quantity
- Real-time variance calculation
- Historical consumption pattern display
- Stock availability validation
- Multi-step submission progress indicator

Component Properties

Property	Type	Description
`OrdemProducao`	`OrdemProducao?`	Production order being processed
`WorkerId`	`int`	Authenticated worker ID
`QuantityProduced`	`decimal`	Actual production quantity
`Waste`	`decimal`	Waste/scrap quantity
`Materials`	`List<MaterialConsumptionDto>`	Material consumption list

Material Loading Strategy

```
private async Task OnQuantityProducedChanged(decimal value)
{
    // Debounce mechanism with 1.5s delay
    _loadMaterialsCts?.Cancel();
    _loadMaterialsCts = new CancellationTokenSource();
    await Task.Delay(1500, token);

    if (!token.IsCancellationRequested)
        await LoadMaterials();
}
```

Submission Process

1. Validate data
2. Update order quantities
3. Generate stock movements
4. Register material consumption
5. Send to Primavera ERP
6. Finalize transaction

Progress Tracking

```
private async Task UpdateProgress(int step, string message)
{
    CurrentStepNumber = step;
    CurrentStep = message;
    StateHasChanged();
    await Task.Yield(); // Allow UI update
}
```

Service Layer

IFactoryFloorService Interface

Core Operations

Production Order Management

```
Task<List<OrdemProducao>> GetProductionOrdersAsync(string? maquinaId =
null);
Task<OrdemProducao?> GetProductionOrderByIdAsync(int orderId,
BioBragaContext? existingContext = null);
```

Production Control

```
Task<(bool success, string message)> StartProductionAsync(int orderId, int
userId);
Task<(bool success, string message)> PauseProductionAsync(int orderId, int
userId);
Task<(bool success, string message)> EndProductionAsync(int orderId, int
userId);
```

Material Management

```
Task<List<MaterialConsumptionDto>> GetMaterialsForProductionAsync(int
orderId, decimal quantityProduced);
Task<(bool success, string message)> RegisterProductionAsync(
    int orderId, int userId, decimal quantityProduced,
    decimal waste, List<MaterialConsumptionDto> actualConsumption);
```

FactoryFloorService Implementation

Dependency Injection

```
public FactoryFloorService(
    IDbContextFactory<BioBragaContext> contextFactory,
    IPrimaveraRepository primaveraRepo,
    ILogger<FactoryFloorService> logger,
    IConfiguration configuration)
```

Material Calculation Algorithm

GetMaterialsForProductionAsync Logic

```
// Core calculation for proportional material consumption
```

```
var totalPlannedForOrder = mat.gf.Quantidade ?? 0;
var orderQuantity = order.Quantidade ?? 1;
var perUnitQuantity = totalPlannedForOrder / orderQuantity;
var proportionalQty = Math.Round(perUnitQuantity * quantityProduced, 6);

// Historical average consideration
decimal? suggestedQty = null;
if (mat.gf.QuantidadeMedia.HasValue && mat.gf.QuantidadeMedia.Value > 0)
{
    suggestedQty = Math.Round(mat.gf.QuantidadeMedia.Value *
quantityProduced, 6);
}
```

Transaction Management

RegisterProductionAsync Transaction Flow

```
using var transaction = await context.Database.BeginTransactionAsync();
try
{
    // 1. Update order quantities
    order.QuantidadeFeita += quantityProduced;
    order.QuantidadeFalta -= quantityProduced;

    // 2. Update consumption tracking
    gastoFase.QuantidadeConsumida += material.ActualQuantity;
    gastoFase.NumeroProducoes++;

    // 3. Generate stock movements
    await GenerateStockMovementsAsync(...);

    // 4. Commit transaction
    await transaction.CommitAsync();
}
catch
{
    await transaction.RollbackAsync();
    throw;
}
```

Stock Movement Generation

Document Generation Pattern

1. **Stock Entry (ENTSTOCK):** Records finished goods
2. **Stock Exit (SAIDASTOCK):** Records material consumption
3. **Waste Document (DESP):** Records production waste

```
private async Task GenerateStockEntryAsync(
    BioBragaContext context,
    OrdemProducao order,
    int userId,
    decimal quantity)
{
    // Document header creation
    var doc = new CrmDocumento
    {
        Tipodocumentoid = tipoDoc.Id,
```

```
        Numero = await GetNextDocumentNumber(),
        Data = DateTime.Now,
        DocumentoOrdemFabrico = order.op.Id
    };

    // Document lines for product and reference
}
```

Data Models

Core DTOs

MaterialConsumptionDto

```
public class MaterialConsumptionDto
{
    public int GastosFaseId { get; set; }
    public int? ArtigoId { get; set; }
    public string ArtigoNome { get; set; }
    public string ArtigoCodigo { get; set; }
    public string Unidade { get; set; }

    // Quantities
    public decimal PlannedQuantity { get; set; }
    public decimal ActualQuantity { get; set; }
    public decimal Stock { get; set; }

    // Variance Analysis
    public decimal Variance => ActualQuantity - PlannedQuantity;
    public decimal VariancePercentage =>
        PlannedQuantity != 0 ? (Variance / PlannedQuantity) * 100 : 0;

    // Historical Data
    public decimal? HistoricalAverage { get; set; }
    public decimal? TotalConsumed { get; set; }
    public int? ProductionCount { get; set; }
}
```

ProductionStatsDto

```
public class ProductionStatsDto
{
    public decimal? TotalProduced { get; set; }
    public decimal? TotalPending { get; set; }
}
```

Entity Models

OrdemProducao (Composite Model)

```
public class OrdemProducao
{
    public CrmOrdemProducao op { get; set; }
    public CrmDocumentoDetalhe docline { get; set; }
    public CrmDocumento doc { get; set; }
    public CrmArtigo artigo { get; set; }
    public List<ComponenteProd> ListaComponentes { get; set; }
}
```

API Reference

Service Registration

```
// Program.cs
services.AddScoped<IFactoryFloorService, FactoryFloorService>();
services.AddDbContextFactory<BioBragaContext>(options =>
    options.UseSqlServer(connectionString));
```

Endpoint Patterns

- `/factory_new` - General production view
- `/factory_new/{maquinaid}` - Machine-specific view

State Management

Component State Flow

```
User Action → Component Event → Service Call → Database Update → UI Refresh
     ↑                                                                ↓
     └──────────────── StateHasChanged() ←──────────────────────────┘
```

Cascading Parameters

```
[CascadingParameter] public CrmUtilizador? CurrentWorker { get; set; }
[CascadingParameter] public EventCallback<CrmUtilizador?> OnWorkerChanged {
get; set; }
```

State Persistence

- URL-based state for machine selection
- Session-based worker authentication
- Database-persisted production state

Integration Points

Primavera ERP Integration

```
await _primaveraRepo.SendMovimentacaoStock(doc, lines);
```

Database Context Management

```
// Context reuse pattern for transactions
var shouldDisposeContext = existingContext == null;
var context = existingContext ?? await
_contextFactory.CreateDbContextAsync();
try
{
    // Operations
}
finally
{
    if (shouldDisposeContext)
        await context.DisposeAsync();
}
```

Security

Authentication Flow

1. PIN-based worker authentication
2. Hashed PIN storage (`Utils.Hash(pin)`)
3. Active user validation
4. Action-level authentication requirements

Authorization Checks

```
if (CurrentWorker == null) return;
if (worker != null && worker.Isactive != true) return null;
```

Audit Trail

- All actions logged with user ID and timestamp
- Production logs stored in `CrmOrdemProducaoLog`
- Document creation tracked with `Criadopor` field

Performance Considerations

Optimization Strategies

Debouncing

```
// 1.5-second debounce for material loading
await Task.Delay(1500, token);
```

Eager Loading

```
var query = from op in context.CrmOrdemProducaos
            join docDetail in context.CrmDocumentoDetalhes...
            // Multiple joins for complete data
```

Local Filtering

```
private void FilterOrdersByMachine()
{
    // Client-side filtering to avoid server roundtrip
    filteredProductionOrders = allProductionOrders
        .Where(o => o.op.Equipamentoid.ToString() == maquinaid)
        .ToList();
}
```

Scalability Considerations

- Database connection pooling via `IDbContextFactory`
- Asynchronous operations throughout
- Transaction scope minimization
- Indexed database queries on frequently accessed fields

Monitoring Points

- Production order query performance
- Material calculation response time
- Stock movement generation duration
- Primavera integration latency

Error Handling

Service Layer Pattern

```
try
{
    // Operation
    return (true, "Success message");
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error description");
    return (false, $"Error message: {ex.Message}");
}
```

UI Error Feedback

```
Snackbar.Add($"Error message", Severity.Error);
```

Deployment Considerations

Configuration Requirements

- Connection strings for BioBraga database
- Primavera API endpoints
- Logging configuration
- Image storage path configuration

Environment Variables

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=...;Database=BioBraga;..."
  },
  "Primavera": {
    "ApiEndpoint": "https://..."
  }
}
```

Health Checks

- Database connectivity
- Primavera API availability
- Service layer responsiveness

Maintenance and Support

Logging Strategy

- Structured logging with correlation IDs
- Performance metrics for key operations

- Error tracking with stack traces
- User action audit trail

Database Maintenance

- Regular index optimization
- Archive historical production logs
- Stock level reconciliation procedures
- Cleanup of cancelled/completed orders

Version Control

- Semantic versioning for releases
- Database migration scripts
- Backward compatibility considerations
- Feature flag implementation for gradual rollouts